



Security Review For Mezzanine



Collaborative Audit Prepared For: **Mezzanine**
Lead Security Expert(s): **0x73696d616f**
Date Audited: **March 20 - March 21, 2026**

Introduction

Mezzanine is a structured yield protocol built around a three-tranche system. Users deposit USDC to mint mzUSD, which can be held as a liquid base asset or staked into one of three yield-bearing tranches: Senior (smzUSD), Mezzanine (MLT), and Junior (JLT).

Each tranche represents a defined position in the loss waterfall. JLT absorbs losses first and receives the highest risk premium. MLT sits in the second loss position. smzUSD absorbs losses last and earns base yield with no risk premium. Capital is deployed into whitelisted DeFi strategies via institutional MPC custody, with all strategy wallets publicly visible on-chain.

Scope

Repository: [woof-software/mezzanine-sc](https://github.com/woof-software/mezzanine-sc)

Audited Commit: [7c05a6c328462ddd7cd44db2814267350e6438f2](#)

Final Commit: [be773e3c7aa0c0cab42e1b3d169883ee1154c19d](#)

Files:

- [contracts/Allowlist.sol](#)
- [contracts/MZUSD.sol](#)

Final Commit Hash

[be773e3c7aa0c0cab42e1b3d169883ee1154c19d](#)

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	0	2

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue L-1: Allowlist::initialize() does not emit an event when setting the sanctionsList

Source: <https://github.com/sherlock-audit/2026-03-mezzanine-update-mar-20th-2026/issues/3>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Allowlist::initialize() sets the sanctionsList state variable but does not emit an event.

Vulnerability Detail

In `Allowlist::initialize()`, the `sanctionsList` is set without emitting an event:

```
function initialize(
    address _accessManager,
    ISanctionsList _sanctionsList
) external initializer {
    require(address(_sanctionsList) != address(0), ZeroAddress());
    sanctionsList = _sanctionsList; //@audit ISSUE missing event
    __AccessManaged_init(_accessManager);
}
```

The `sanctionsList` is the Chainalysis sanctions oracle used in `verifyAccount()` to block sanctioned addresses.

Impact

Missing event

Code Snippet

<https://github.com/sherlock-audit/2026-03-mezzanine-update-mar-20th-2026/blob/main/mezzanine-sc/contracts/Allowlist.sol#L32>

Tool Used

Manual Review

Recommendation

Emit an event when the `sanctionsList` is set in `initialize()`:

```
event SanctionsListSet(address indexed sanctionsList);
```

```
function initialize(
    address _accessManager,
    ISanctionsList _sanctionsList
) external initializer {
    require(address(_sanctionsList) != address(0), ZeroAddress());
    sanctionsList = _sanctionsList;
+   emit SanctionsListSet(address(_sanctionsList));
    __AccessManaged_init(_accessManager);
}
```

Issue L-2: Allowlist does not call `_disableInitializers()` in the constructor

Source: <https://github.com/sherlock-audit/2026-03-mezzanine-update-mar-20th-2026/issues/4>

Summary

Allowlist is a UUPS upgradeable contract that lacks a constructor calling `_disableInitializers()`.

Vulnerability Detail

The `Allowlist` contract inherits `UUPSUpgradeable` and `AccessManagedUpgradeable`, but has no constructor:

```
contract Allowlist is AccessManagedUpgradeable, UUPSUpgradeable {
    ISanctionsList sanctionsList;
    mapping(address => bool) public isAllowlisted;
    // ... no constructor calling _disableInitializers()
```

Without `_disableInitializers()` in the constructor, the implementation contract itself is left uninitialized. An attacker calls `initialize()` on the implementation, passing their own `_accessManager`. Since `_authorizeUpgrade()` is gated by `restricted`, and the attacker now controls the `AccessManager` on the implementation, they have full control over the implementation contract.

Impact

No impact to the real contract (proxy), but it may add phishing risks in the implementation contract.

Code Snippet

<https://github.com/sherlock-audit/2026-03-mezzanine-update-mar-20th-2026/blob/main/mezzanine-sc/contracts/Allowlist.sol#L12>

Tool Used

Manual Review

Recommendation

Add a constructor that calls `_disableInitializers()`:

```
+/// @custom:oz-upgrades-unsafe-allow constructor
+constructor() {
+  _disableInitializers();
+}
```

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.